



VASTRA

- INDIAN OUTFIT BUILDER

ASSIGNMENT - 2



SUBMITTED TO
FACULTY - S DURGA DEVI

SUBJECT & CODE - WEB PROGRAMMING | 22CSC42
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY (A)

SUBMITTED BY
VUNDELA KEERTHI REDDY | ROLL NO: 160124733023
VUPPU ABHIGNA | ROLL NO: 160124733024

1. PROBLEM STATEMENT

In today's fashion and online shopping environment, users are exposed to a wide variety of clothing options across multiple categories such as ethnic wear, western wear, accessories, and footwear. However, most platforms display these items individually without helping users understand how different pieces can be combined to form a complete outfit.

Users often struggle to decide:

- Which top matches which bottom
- What accessories suit a particular outfit
- Whether selected items look good together as a whole

This leads to confusion, poor outfit planning, & increased time spent in decision-making.

Key problems addressed:

- **No outfit visualization:** Existing platforms do not allow users to see a complete outfit before purchasing.
- **Disconnected product browsing:** Items are shown separately without guidance on combining them.
- **Manual decision-making:** Users must rely entirely on their own judgment without system support.

Solution:

The **VASTRA – Indian Outfit Builder** is designed to solve these problems by providing an interactive platform where users can:

- Browse categorized clothing items
- Select and combine items into outfit slots
- Visualize a complete outfit in real time
- Manage selections through wishlist and cart

This system simplifies outfit planning and enhances user experience by making fashion selection more intuitive and interactive.

2. ABSTRACT

VASTRA is a dynamic web-based outfit builder application that allows users to create personalized outfits by selecting clothing items from a structured product catalog.

How it works step by step:

1. Product Loading:

When the application starts, product data is fetched from a backend JSON server. This data includes attributes such as name, category, price, rating, and tags.

2. Catalog Display:

All products are displayed in a categorized format including Ethnic Tops, Western Tops, Bottoms, Footwear, Bags, Jewelry, and Outerwear.

3. Search & Filter:

Users can search for items or filter them based on categories to quickly find relevant products.

4. Outfit Builder:

The center section allows users to add selected items into specific slots such as:

- Top/Kurti
- Bottom
- Footwear
- Accessories

5. Dynamic Updates:

As users select items, the outfit is updated instantly using Vue.js reactive properties.

6. Total Calculation:

The system automatically calculates the total price of selected items.

7. Wishlist & Cart:

Users can save items to wishlist or add them to cart for further actions.

The frontend is built using Vue.js for dynamic UI updates, while JSON Server acts as a backend to provide real-time data. The system demonstrates how modern web technologies can be used to create an interactive and responsive fashion application.

3. TECHNOLOGIES USED

3.1 Frontend Technologies

- **HTML5:**
Used to structure all sections of the application such as product catalog, outfit builder, wishlist, and cart.
- **CSS3:**
Handles complete UI design including layout, colors, animations, responsive design, and styling of product cards and builder sections.
- **JavaScript(ES6):**
Implements application logic such as item selection, filtering, search functionality, and event handling.
- **Vue.js:**
Used for building reactive UI. It automatically updates the interface when data changes and manages state efficiently.

3.2 Backend Technologies

- **JSON Server:**

Acts as a lightweight backend that provides REST API endpoints like:

```
/products  
/cart  
/wishlist
```

It allows fetching and updating data without creating a full backend server.

3.3. Data Storage

- **JSON File:**

Stores product data including: **name, category, price, rating, & tags.**
This structured format makes it easy to manage and retrieve data.

3.4 Development Tools

- VS Code (Code Editor)
- Browser Developer Tools
- Command Prompt / PowerShell (to run backend)

4. SYSTEM ARCHITECTURE

4.1 Overall Structure

The system follows a simple architecture:

- Frontend (Vue.js UI)
- Backend (JSON Server)
- Database (JSON file)

The frontend communicates with backend using HTTP requests.

4.2 Product Flow

1. Application loads
2. Frontend sends request → `/products`
3. Backend returns JSON data
4. Vue.js stores data in `catalog`
5. UI displays products dynamically

4.3 Outfit Builder Logic

- Each outfit slot represents a category
- When user clicks “Add”:
 - Item is assigned to that slot
- If slot already filled → replaced

4.4. Dynamic Updates

Using Vue.js:

- UI updates instantly
- No page reload required
- Changes reflect automatically

4.5 Filtering & Search

- Category buttons filter products
- Search bar filters by name
- Sorting allows better browsing

4.6 Cart & Wishlist Working

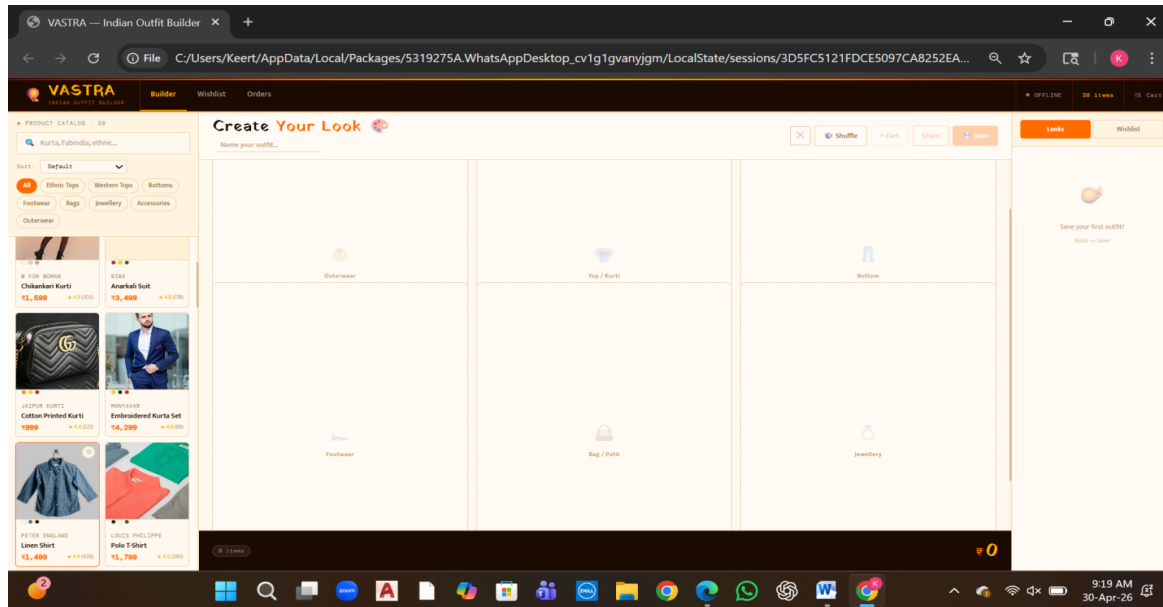
- Selected items stored temporarily
- User can:
 - Save items
 - View later
 - Add/remove dynamically

4.7 Key Implementation Features

- ✓ Reactive UI (Vue.js)
- ✓ API-based data fetching
- ✓ Real-time updates
- ✓ Structured product categories
- ✓ Interactive outfit builder

5. OUTPUT SCREENSHOTS

Builder/Home Page:

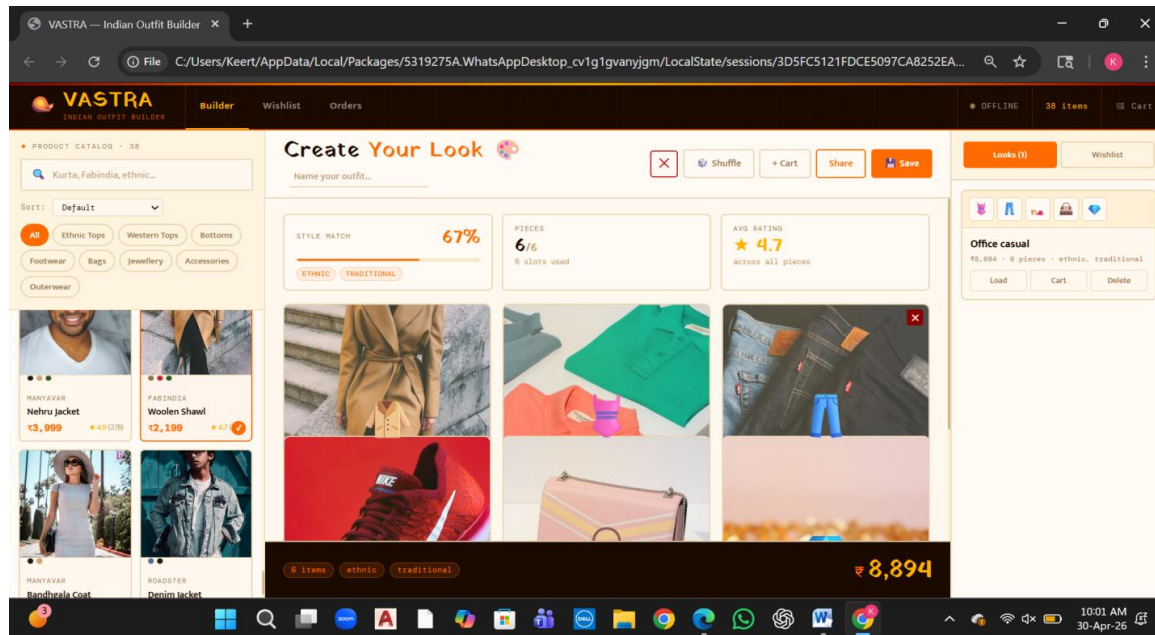


This home (builder) page of VASTRA serves as an interactive outfit creation interface where users can design their own looks. On the left side, a product catalog is displayed with search and filter options such as Ethnic Tops, Western Tops, Bottoms, Footwear, and Accessories, allowing users to easily browse items. Each product card shows an image, brand name, price, and rating.

The center section is the main workspace titled “Create Your Look,” where users can build an outfit by adding items into different categories like Top/Kurti, Bottom, Footwear, Jewelry, Bag, and Outerwear. This section is designed to visually organize selected items and simulate a complete outfit.

On the right side, there is a panel for saved looks or wishlist, helping users store and revisit their outfit combinations. At the bottom, the total price of the selected items is displayed, giving users a clear idea of the overall cost.

Create & Save Looks:



The **Create & Save Look** feature in the **VASTRA** home page allows users to design and store complete outfit combinations in an interactive way.

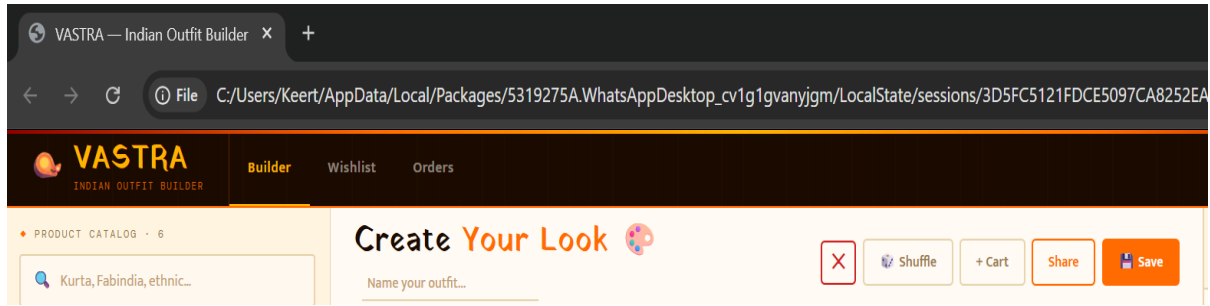
In the “**Create Your Look**” section, users can build an outfit by selecting items from the product catalog and placing them into predefined categories such as outerwear, top/kurti, bottom, footwear, bag, and jewelry. As items are added, the system dynamically updates key metrics like **style match percentage**, **number of pieces used**, and **average rating**, helping users evaluate how well their outfit fits together in terms of style and quality.

Users can also name their outfit using the input field provided, making it easier to identify and organize multiple looks. Additional controls like **Shuffle** (for random suggestions) and **Cart** (to add selected items for purchase) enhance usability.

Once the outfit is complete, the **Save** option allows users to store the look in the “Looks” section on the right panel. Saved looks display a preview of selected items along with total price and style tags (such as casual, minimal, western, or ethnic). Users can later **load**, **edit**, **add to cart**, or **delete** these saved combinations.

Overall, this feature enables users to experiment with different styles, evaluate outfit combinations, and conveniently save and manage their personalized fashion looks.

More Options:



This section is the **header of the outfit builder workspace** in the VASTRA application.

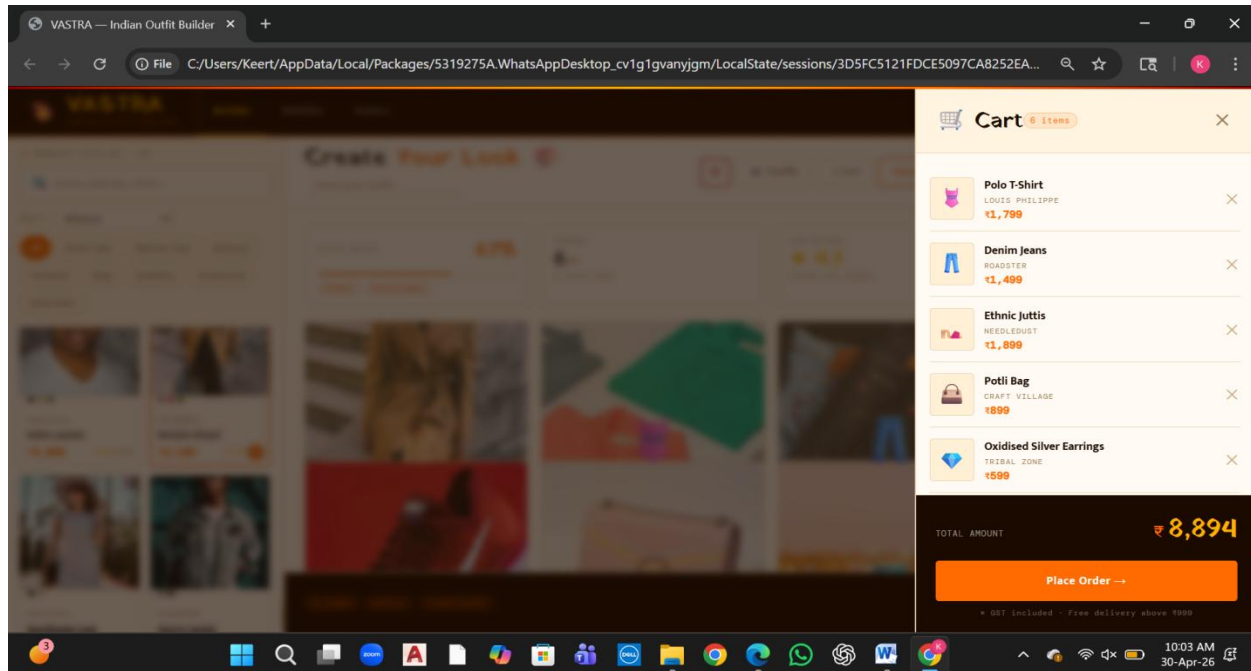
It displays the title “**Create Your Look**”, indicating that the user is in the outfit creation area. Below the title, there is an input field labeled “**Name your outfit...**”, which allows users to assign a custom name to their designed look for easy identification and saving.

On the right side, several action buttons are provided:

- **Close (X)** – clears or exits the current outfit
- **Shuffle** – suggests a random combination of items
- **+ Cart** – adds the selected outfit items to the cart
- **Share** – enables sharing the created look
- **Save** – stores the outfit in the saved looks section

Overall, this header acts as the control panel for managing, customizing, and saving the outfit being created.

Cart & Placing the Order:



The **Cart** section in the VASTRA application provides a detailed summary of all the items selected by the user for purchase.

It appears as a side panel displaying a list of added products, where each item includes its **image icon, name, brand, and price**. Users can easily review their selections such as clothing, footwear, and accessories in one place. Each item also has a **remove (x) option**, allowing users to delete products directly from the cart.

At the bottom of the cart, the **total amount** of all selected items is clearly calculated and displayed, giving users a complete cost overview. A prominent **“Place Order”** button is provided to proceed with the purchase. Additional information like GST inclusion and delivery conditions is also mentioned for transparency.

Overall, the cart is designed to be simple and user-friendly, enabling users to manage their selected items, make quick modifications, and proceed to checkout efficiently.